

A Reinforcement Learning and Prediction-Based Lookahead Policy for Vehicle Repositioning in Online Ride-Hailing Systems

Honghao Wei¹, Member, IEEE, Zixian Yang², Xin Liu², Zhiwei Qin², Member, IEEE, Xiaocheng Tang, and Lei Ying², Fellow, IEEE

Abstract—Existing approaches for vehicle repositioning on large-scale ride-hailing platforms either ignore the spatial-temporal mismatch between supply and demand in real-time or overlook the long-term balance of the system. To account for both, we propose a lookahead repositioning policy in this paper, which is a novel approach to repositioning idle vehicles from both a dynamic system and a long-term performance perspective. Our method consists of two parts; the first part utilizes linear programming (LP) to formulate the nonstationary system as a time-varying, T -step lookahead optimization problem and explicitly models the fraction of drivers who follow repositioning recommendations (called the repositioning rate). The second step is to incorporate a reinforcement learning (RL) method to maximize long-term return based on learned value functions after the T time slots. Extensive studies utilizing a real-world dataset on both small-scale and large-scale simulators show that our method outperforms previous baseline methods and is robust to prediction errors.

Index Terms—Ride-hailing, large-scale, reposition, idle car routing, reinforcement learning.

I. INTRODUCTION

THE past decade has seen a rapid proliferation of ride-sharing services. Large-scale ride-hailing systems, such as Uber, DiDi, and Lyft, have fundamentally transformed lives and the way we travel in cities. The rising prevalence of these ride-hailing platforms raises an important operational question, how to provide a reliable means of transportation while fulfilling most requests? This challenge is particularly pertinent in a highly dynamic environment characterized by a temporal and spatial imbalance between the supply of drivers and the demand from passengers. To address this imbalance issue, one possible approach involves the adoption of a

centralized planner, which proactively communicates repositioning suggestions to idle cars, aiming to relocate them strategically in anticipation of future demand shortages at specific destinations. Implementing such a strategy holds the potential to substantially augment driver incomes while concurrently enhancing the overall passenger experience.

Existing works like [1], [2], and [3] formulated the repositioning task into a fluid-based optimization problem that can be solved by using LP, which captures the first-order statistics of the complex system. However, a common assumption made in these works is that the system is already in a steady state, implying that it has reached a stable equilibrium. Additionally, they assume that model parameters, such as the arrival rate at each location, are known. These assumptions, though convenient for analysis, are often unrealistic in practical ride-hailing systems. Another line of related work is the model predictive control (MPC) approach [4], [5], [6], which leverages short-term demand forecasts based on historical data to obtain repositioning strategies by solving a planning problem. However, both the existing LP and MPC methods assume all the vehicles will follow the repositioning recommendations, which may not be realistic in ride-hailing systems.

Reinforcement learning (RL) has also been used in ride-hailing systems for order dispatching [7], [8] and repositioning [9], [10], [11], [12]. For example, [11] proposed an RL method to learn a driver-perspective state-value function using spatiotemporal deep value networks and to generate repositioning actions through value-based policy search. References [9] and [10] proposed deep RL approaches, such as deep Q-networks (DQN) and proximal policy optimization (PPO), for vehicle repositioning. These approaches did not explicitly consider the dynamics of the system. For example, crowding a large fleet of vehicles into a single location with high value causes undesirable imbalance. As shown in [12], the performance of the RL-based approach decreases when the repositioning rate of vehicles becomes larger. A comprehensive survey on reinforcement learning for ride-sharing systems can be found in [13].

As we mentioned above, we first need to consider the nonstationarity of the system. Additionally, the algorithm should model the repositioning rate precisely. This factor is critical when designing repositioning algorithms. It recognizes the reality that drivers in the system have their own decision-making autonomy and may not always strictly follow the suggested repositioning strategies. Drivers deviating from

Manuscript received 18 December 2022; revised 24 May 2023 and 28 August 2023; accepted 1 September 2023. Date of publication 14 September 2023; date of current version 2 February 2024. This work was supported in part by the DiDi Research partnership with the University of Michigan. The Associate Editor for this article was G. Guo. (Corresponding author: Honghao Wei.)

Honghao Wei is with EECS, Washington State University, Pullman, WA 99164 USA (e-mail: honghao.wei@wsu.edu).

Zixian Yang and Lei Ying are with EECS, University of Michigan, Ann Arbor, MI 48109 USA.

Xin Liu is with the School of Information Science and Technology, ShanghaiTech University, Shanghai 201210, China.

Zhiwei Qin was with the Laboratory of DiDi, Mountain View, CA 94043 USA. He is now with the Lyft Rideshare Laboratories, San Francisco, CA 94107 USA.

Xiaocheng Tang was with the Laboratory of DiDi, Mountain View, CA 94043 USA. He is now with the Meta AI, San Francisco, CA 94102 USA.

Digital Object Identifier 10.1109/TITS.2023.3312048

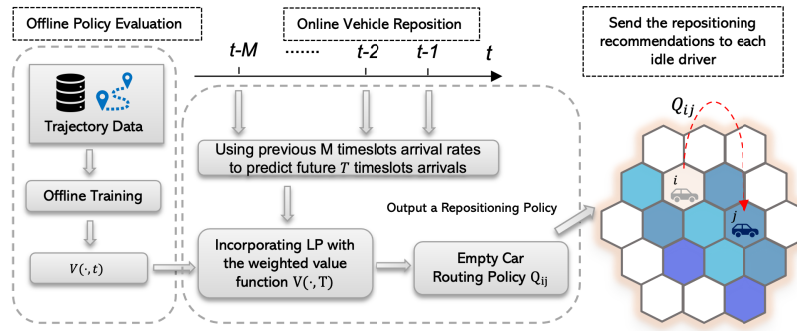


Fig. 1. An illustration of our method on empty car repositioning.

repositioning recommendations can significantly impact the overall performance of the ride-hailing system. The decisions made by individual drivers regarding their repositioning actions can lead to changes in demand-supply dynamics, affecting wait times, service quality, and overall system efficiency.

In light of these limitations, in this paper, we propose a practical, data-driven lookahead policy considering short-term system dynamics and long-term return simultaneously. As illustrated in Fig. 1, for each timeslot t , our method consists of two phases, offline policy evaluation (OPE) and online vehicle repositioning. In OPE phase (details are described in Section III), We first adopt the approach proposed in [7] for training the value function V for each region and timeslot from the abundant offline historical driver trajectories date. In the online vehicle repositioning phase, by incorporating the weighted trained value function, we use an LP to formulate the online idle car repositioning problem with a predicted lookahead time horizon T , which explicitly models the repositioning rate of vehicles and also the nonstationarity of the system (details in Section II). Then a real-time repositioning policy Q_{ij} is achieved by solving the LP. Finally, the centralized planner sends the repositioning recommendations to each idle driver and the system moves to the next timeslot.

The main contributions of our paper are summarized below:

- Our proposed LP formulation considers a more realistic and reasonable situation where only a subset of idle drivers accept the repositioning recommendations.
- Considering the nonstationarity of the online ride-hailing system, we relax the flow balance equation, which makes the solution of our LP more robust. Our algorithm can achieve the same performance as the case without considering nonstationarity with only one-sixth of the repositioning rate.
- The optimal solution at each time optimizes the order completion rate over T time slots while also capturing the system's future rewards by including the pre-trained value function in the objective function.
- The policy works well with the predicted arrival rate when the true arrival rates are unknown because our prediction method captures the spatiotemporal correlation between regions.
- The majority of existing formulations employ a grid-based system and assume that drivers only accept orders

in their present grids. In a real system, the platform dispatches orders based on the distance between drivers and customers (broadcasting distance) instead of dividing a region into a grid. We propose a splitting trick to mitigate this discrepancy.

- Simulation results based on a real-world dataset demonstrate that our method outperforms the state-of-the-art idle car repositioning algorithms.

We finally would like to remark on some key observations from our numerical study. First, it has been observed that even with a repositioning rate of 0.12, i.e., only 12% of idle drivers follow the repositioning recommendations, our proposed LP-based lookahead policy can achieve completion and utility rates that are higher than 0.87. Based on this key observation, where only a small number of drivers following repositioning recommendations can still let the online platform achieve well performance highlights the potential for a more flexible incentive approach. As a result, there may be a reduced need for the platform to actively encourage or enforce all drivers to accept these recommendations. Furthermore, the numerical results also show that repositioning provides the most significant improvement in the completion rate when the gap between the supply (the number of drivers), and the demand (the number of ride requests) is not too large and not too small. Furthermore, including trip costs in the LP's objective function further lowers travel costs while maintaining a high completion rate.

II. PROBLEM FORMULATIONS

In this section, we formally introduce our formulation of the repositioning problem.

A. Lookahead LP With Value Function

Motivated by [1] and [14], We also model this ride-sharing network with a closed queuing network consisting of both single-server and infinite-server stations. The cars in this system are considered as “jobs” through the queueing network. In particular, cars that are awaiting passengers in a particular region are characterized by a single-server station. The number of cars waiting in the region is represented by the buffer of this station, while passenger ride requests are treated as service completions at the station. Consequently, the service times at the single-server station correspond to the inter-arrival

TABLE I
NOTATION TABLE

Notation	Description
$\bar{e}_{ij}$	Average number of cars driving empty from region i to region j .
$\bar{f}_{ij}$	Average number of cars driving with customers from region i to region j .
η	Repositioning rate.
$\bar{a}_i$	Order completion rate in region i .
$\lambda_i(t)$	Number of passengers arriving in region i at time t .
$1/\mu_{ij}(t)$	Travel time from region i to region j at time t .
$q_{ij}(t)$	Empty cars routing probability from region i to region j at time t .
$P_{ij}(t)$	The probability that cars travel from region i to region j at time t .

TABLE II
MARKOV CHAIN TRANSITION RATES

Rate	Description
$N\lambda_i(t)P_{ij}(t)$	Passenger arrives at region i and starts ride from i to j at time t .
$\mu_{ij}(t)\bar{f}_{ij}\eta q_{jk}(t)$	Passenger starts from region i , dropped off at region j . Car follows the repositioning recommendation, and starts driving empty to region k with probability η .
$\mu_{ji}(t)\bar{f}_{ji}(\eta q_{ii}(t) + 1 - \eta)$	Passenger starts from region j , dropped off at region i . Car stays at region i .
$\mu_{ij}(t)\bar{e}_{ij}$	Empty car travels from region i to region j and stays there.

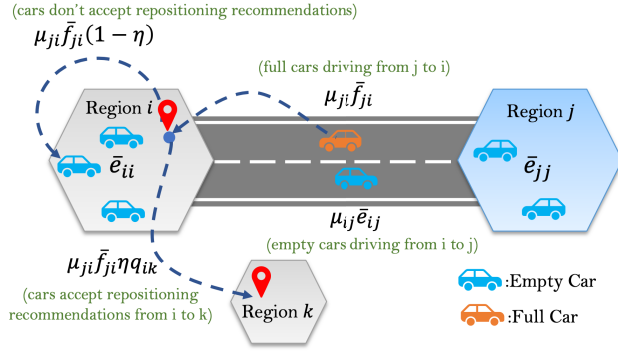


Fig. 2. The queuing Structure of the ride-sharing network representing traveling vehicles from regions.

times of passengers to the region, despite the absence of a physical “server” at the station. The infinite-server stations are used to model car travel between regions which means a car immediately heads to the destination when an order is accepted. The driver does not need to wait since the number of servers is assumed to be infinite.

Let N be the number of vehicles in the city, and r be the number of regions. The number of passengers arriving in area i at time slot t is assumed to be Poisson distributed with parameter $N\lambda_i(t)$, $i \in [r]$, where $i \in [r]$ stands for $i \in \{1, 2, \dots, r\}$. We summarize the notations used in our model in Table I. For any $i \in [r]$, $j \in [r]$, let $1/\mu_{ij}(t)$ denote the travel time from region i to region j at time slot t . Let $P_{ij}(t)$ denote the order destination distribution at time slot t , i.e., the probability of the event that a passenger travels to region j given that the passenger request originates from region i . Let $q(t) = \{q_{ij}(t)\}$ be our repositioning policy at time slot t , where idle cars in region i are relocated with probability $q_{ij}(t)$ to region j . Our goal is to discover an optimal solution $q^*(t)$ that maximizes the lowest completion rate of all ride requests across all regions.

We remark that there are several key differences between our formulation and the one in [1]. Firstly, we explicitly model the fraction of drivers that follow repositioning recommendations in the formulation while they assume that all drivers will

accept the repositioning recommendations. Secondly, we relax the constraint in the LP formulation to model nonstationary systems while [1] focuses on the steady-state performance. Thirdly, the max-min objective used in our formulation turns out to be more robust to noises based on our numerical study. More discussions and details of the objective function can be found in Section VI-D.

Given all the parameters, the continuous time Markov chain of the queuing model is described in Table II. According to the definitions above, our LP formulation with multi-step lookahead and learned value functions is presented in (1a)-(1f):

$$\max_{\bar{a}, \bar{f}, \bar{e}} \left(\min_i \bar{a}_i + w_v \sum_j V(j, t+T) \sum_i (\bar{e}_{ij} + \bar{f}_{ij}) \right) \quad (1a)$$

subject to

$$\sum_{u=0}^{T-1} \lambda_i(t+u) P_{ij}(t+u) \bar{a}_i = \sum_{u=0}^{T-1} \mu_{ij}(t+u) \bar{f}_{ij}, \quad (1b)$$

$$\sum_{u=0}^{T-1} \mu_{ij}(t+u) \bar{e}_{ij} \leq \eta \sum_{k=1}^r \sum_{u=0}^{T-1} \mu_{ki}(t+u) \bar{f}_{ki}, \quad i \neq j \quad (1c)$$

$$\sum_{u=0}^{T-1} \lambda_i(t+u) \bar{a}_i + \sum_{j=1, j \neq i}^r \sum_{u=0}^{T-1} \mu_{ij}(t+u) \bar{e}_{ij} \leq \sum_{k=1, k \neq i}^r \sum_{u=0}^{T-1} \mu_{ki}(t+u) \bar{e}_{ki} + \sum_{k=1}^r \sum_{u=0}^{T-1} \mu_{ki}(t+u) \bar{f}_{ki}, \quad (1d)$$

$$\sum_{i=1}^r \sum_{j=1}^r (\bar{e}_{ij} + \bar{f}_{ij}) = 1 \quad (1e)$$

$$\bar{e}_{ij} \in [0, 1], \bar{f}_{ij} \in [0, 1], \bar{a}_i \in [0, 1], \forall i, j \in [r] \quad (1f)$$

Note that the parameters $\lambda_i(t)$, $P_{ij}(t)$, and $\mu_{ij}(t)$ are time-varying. Hence, we use a T -time-slot lookahead method, which uses the parameters averaged over the time slots $t, t+1, \dots, t+T-1$ to generate a repositioning policy that takes the future variation into account.

In the objective function (1a), the first term $\bar{a} = \{\bar{a}_i\}$ stands for the order completion rates in all regions. Hence, the first term maximizes the completion rate of the most severe region in short supply.

The second term of (1a) represents the expected discounted total returns from time $t+T$ to the end of the time horizon (i.e. the end of the day) we consider. In this term, w_v is a weight used to strike a balance between the order completion rate over T time slots and the future rewards after time slot T . In particular, $V(j, t)$ is the value function for a vehicle in region j at time slot t , defined as

$$V(j, t) = \sum_{k=t}^{T_{\text{end}}} \mathbb{E} \left[\gamma^{k-t} R_k | j, t \right] \quad (2)$$

where T_{end} is the last time slot of the time horizon, the pair (j, t) is regarded as the state for the value function, and γ is the discount factor. R_k is the reward that the vehicle obtains at time slot k . The reward of an order is assumed to be spread uniformly across the trip duration. $\bar{e} = \{\bar{e}_{ij}\}$ and $\bar{f} = \{\bar{f}_{ij}\}$ are the average number of empty vehicles and full vehicles driving from region i to region j , respectively.

In the constraint (1b), the left-hand side $\sum_{u=0}^{T-1} \lambda_i(t+u)P_{ij}(t+u)\bar{a}_i$ is the total expected inflow of occupied vehicles from region i to region j (infinite server station), which equals the right-hand side $\sum_{u=0}^{T-1} \mu_{ij}(t+u)\bar{f}_{ij}$, i.e., the total expected outflow of occupied vehicles (cars with passengers). The constraint (1e) is a normalization for the sum of $\bar{e}_{ij}$ and $\bar{f}_{ij}$, which is the total number of vehicles. The constraints (1f) ensure that $\bar{e}_{ij}$ and $\bar{f}_{ij}$ are non-negative and $\bar{a}_i \in [0, 1]$. The constraints (1c) and (1d) will be illustrated in the following subsection.

B. Repositioning Rate Formulation and Relaxed Constraints

In an idealized system, we wish every driver follows the repositioning recommendations from the centralized planner. However, in practice, drivers make their own decisions and have their own preferences, so only a fraction of them will follow the repositioning recommendations.

In the LP formulation, we introduce a parameter, repositioning rate $\eta \in (0, 1]$, which represents the probability that a driver accepts the repositioning recommendation. We further assume that if a driver denies the recommendation, the idle car remains in the current location. Hence, we have the following two flow balance equations:

$$\begin{aligned} \sum_{u=0}^{T-1} \mu_{ij}(t+u)\bar{e}_{ij} &= q_{ij}\eta \sum_{k=1}^r \sum_{u=0}^{T-1} \mu_{ki}(t+u)\bar{f}_{ki}, i \neq j \quad (3a) \\ \sum_{u=0}^{T-1} \lambda_i(t+u)\bar{a}_i &= \sum_{k=1, k \neq i}^r \sum_{u=0}^{T-1} \mu_{ki}(t+u)\bar{e}_{ki} \\ &\quad + (1 - \eta + \eta q_{ii}) \sum_{k=1}^r \sum_{u=0}^{T-1} \mu_{ki}(t+u)\bar{f}_{ki}, \end{aligned} \quad (3b)$$

where the term $q_{ij}\eta$ represents the probability that an empty vehicle is repositioned from region i to region j , and the term

$(1 - \eta + \eta q_{ii})$ represents the probability that an empty vehicle stays in the current region i . Hence, the right-hand side of the constraint (3a) is the total inflow of empty vehicles on the way from region i to region j (by repositioning), which equals the left-hand side, i.e., the total expected outflow of empty vehicles. Since $q_{ij} \leq 1$, the equality (3a) implies the inequality constraint (1c) in our formulation (1). As for (3b), the left-hand side is the total expected outflow of vehicles from region i and the right-hand side is the total expected inflow of vehicles to region i .

Note that the original LP in [1] was formulated under the assumption that the Markov chain is already in a steady state. However, the system usually converges to a steady state very slowly. Moreover, in practice, when the environment (e.g. traffic pattern) changes fast, the system may rarely be in a steady state. Therefore, in order to capture this nonstationarity, we change the equality in (3b) to an inequality as follows:

$$\begin{aligned} \sum_{u=0}^{T-1} \lambda_i(t+u)\bar{a}_i &\leq \sum_{k=1, k \neq i}^r \sum_{u=0}^{T-1} \mu_{ki}(t+u)\bar{e}_{ki} \\ &\quad + (1 - \eta + \eta q_{ii}) \sum_{k=1}^r \sum_{u=0}^{T-1} \mu_{ki}(t+u)\bar{f}_{ki}, \\ \forall i \in [r] \end{aligned} \quad (4a)$$

which means that we allow the rate of inflow into region i to be larger than the rate of outflow from the region. This may happen when the system is not in a steady state. Since the right-hand side of (3b) is non-increasing in η , the right-hand side may be larger especially when η is small. This idea explains the phenomenon we observed in the simulation that the proposed algorithm has a good performance even if the repositioning rate η is small. Then from (3a), (4a) and the relation $\sum_{j=1}^r q_{ij} = 1$, we can obtain (1d) in our formulation.

III. ALGORITHM

Based on our LP formulation (1a)-(1f), we propose our LP-based and prediction lookahead approach in Algorithm 1. Before using the Algorithm 1, we first need to obtain an estimate of the value function V , for each grid, which can be trained offline by using tabular TD learning. In this paper, we use the discounting technique in [7] and train V offline using historical data. The update rule for the value in the region i at time t in the training process is defined as:

$$\begin{aligned} V^{k+1}(i, t) &\leftarrow V^k(i, t) + \alpha \left[\frac{R_t(\gamma^{L_t} - 1)}{L_t(\gamma - 1)} + \gamma^{L_t} V^k \right. \\ &\quad \left. \times (D_t, t + L_t) - V^k(i, t) \right] \end{aligned} \quad (5)$$

where (R_t, L_t, D_t) is a completed order sampled from historical data with starting region i and starting time slot t . R_t is the reward of the order which is the total fee collected from a trip, L_t is the trip duration, and D_t represents the destination.

After training the value function offline, then at each time slot t , the future arrival rates $\lambda_i(s), s \in \{t, t+1, \dots, t+T-1\}$ are predicted. Next, the arrival rates $\{\lambda_i(s)\}$ will be split to their neighbors according to Algorithm 2 where the splitting trick will be discussed in Section III-B.

Algorithm 1 LP and Prediction-Based Lookahead

- 1 **Input:** $N, \eta, w_v, \mu_{ij}(s), P_{ij}(s),$
 $s \in \{t, t+1, \dots, t+T-1\}, \lambda_i(s),$
 $s \in \{t-M, \dots, t-1\}, V(j, t+T), \forall i, j \in [r]$
- 2 **Output:** $q^*(t)$;
- 3 Predict arrival rates $\lambda_i(s), s \in \{t, \dots, t+T-1\}$;
- 4 $\{\lambda_i(s)\} \leftarrow \text{SPLIT}(\{\lambda_i(s)\})$;
- 5 Solve the LP problem (1), obtain $\bar{e}^*, \bar{f}^*, \bar{a}^*$;
- 6 Compute $q^*(t)$ for $i, j \in [r]$:

$$q_{ij}^*(t) = \frac{\sum_{u=0}^{T-1} \mu_{ij}(t+u) \bar{e}_{ij}^*}{\eta \sum_{k=1}^r \sum_{u=0}^{T-1} \mu_{ki}(t+u) \bar{f}_{ki}^*}, \text{ if } i \neq j \quad (6)$$

$$q_{ii}^*(t) = 1 - \sum_{j \neq i} q_{ij}^*(t), \quad i \in [r] \quad (7)$$

for i **do**

- 7 **if** $q_{ii}^*(t) < 0$ **then**
- 8 $q_{ii}^*(t) \leftarrow 0; \quad q_{\text{sum}} \leftarrow \sum_{j=1}^r q_{ij}^*(t);$
- 9 $q_{ij}^*(t) \leftarrow \frac{q_{ij}^*(t)}{q_{\text{sum}}}, \forall j; \quad // \text{ normalization}$

Algorithm 2 SPLIT

- 1 **Input:** $\{\lambda\}$;
- 2 **Output:** $\{\tilde{\lambda}\}$;
- 3 Initial $\{\tilde{\lambda}\} \leftarrow 0, \alpha_c, \alpha_n$;
- 4 **for** λ_i **in** $\{\lambda\}$ **do**
- 5 $l \leftarrow$ the set of neighbors of i including i ;
- 6 **for** region j **in** l **do**
- 7 $\tilde{\lambda}_j \leftarrow \tilde{\lambda}_j + \lambda_i \cdot \frac{\alpha_c \cdot \mathbb{I}(j=i) + \alpha_n \cdot \mathbb{I}(j \neq i)}{(\text{length}(l)-1)\alpha_n + \alpha_c}.$

Given the approximated value function and the predicted arrival rates, the LP problem is then solved to produce the optimal solution $q^*(t)$ (repositioning matrix) for the current time slot t . The prediction methods and the reason behind splitting will be discussed later.

Note that $q_{ii}^*(t)$ in (7) is not guaranteed to be non-negative. Therefore, we need Line 7-Line 9 to make $q^*(t)$ a valid repositioning policy.

Finally, depending on $q^*(t)$, the centralized planner determines a repositioning policy and transmits the recommendation to each idle driver. This process is repeated at the beginning of every step.

A. Prediction of the Arrival Rates

Note that the proposed algorithm is a lookahead policy, which requires the estimation of future $\lambda_i(s), P_{ij}(s)$, and $\mu_{ij}(s), s \in \{t, t+1, \dots, t+T-1\}$. For the inverse of travel time $\mu_{ij}(s)$ and the destination distribution $P_{ij}(s)$, we estimate them by using the average of the historical data. Since the arrival rates vary significantly during the day and also fluctuate across different days, it is not straightforward to obtain accurate estimations. We use a long short-term memory convolutional neural network (LSTM-CNN) which captures the time series information combined with the detrending

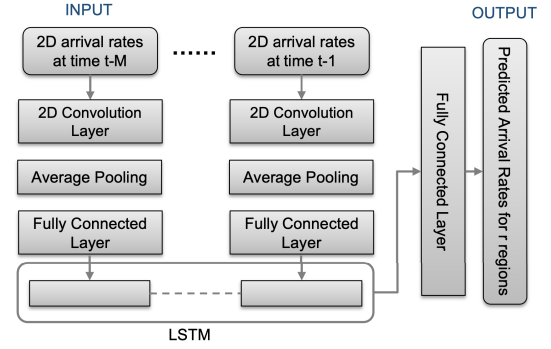


Fig. 3. The structure of LSTM-CNN for future arrival rates prediction.

TABLE III

COMPARISON OF DIFFERENT PREDICTION ALGORITHMS

Algorithm	Testing Mean Square Error
Ridge Regression	8.543
Lasso Regression	8.570
PCR regression	8.661
LSTM-CNN	8.380

method [15], which can reduce the common trend from orders to estimate real-time arrival rates. The input for each CNN component is a 2D grid map which contains the information of the arrival rate of each region. Then a group of CNNs with the information of M previous timeslots arrival rate are considered as input of an LSTM neural network for future prediction. The number of rides requested across different days usually has the same trend during the same time periods, e.g., the number of orders which has peaks during rush hours. The first step of the detrending method is to determine a trend by taking an average of the historical data over multiple days. Then the residual time series can be obtained by subtracting the intra-day trend. After the prediction, the residual arrival rates will be added back to obtain the final predicted future arrival rates. This prediction method efficiently utilizes the spatiotemporal correlation of the arrivals. The structure of LSTM-CNN is shown in Fig. 3, which shares a similar structure with the network in [16]. Therefore, by the elaborate design of the arrival rate predictor, the LSTM part includes the temporal correlation, and the CNN captures the spatial correlation of the arrival rates. The network and training parameters for LSTM-CNN are summarized in Table IV. To demonstrate the advantage of our proposed method for arrival rate prediction, we compare our method with several commonly used regression algorithms and show the results in Table III. We use 30 days of historical data to train the regression model and test them on the same testing dataset. We can observe that our method has the lowest testing error. We remark that in this paper, we focus on traffic prediction based on historical data. However, it's important to note that more factors, such as traffic volume/speed, long distance propagation effect, and anomalous peak events, need to be considered [17], [18], [19].

B. Splitting Arrivals

Our approach also consists of an arrival splitting component, used at Line 4 in Algorithm 1 and defined in Algorithm 2. The LP is formulated on a discretized map (a grid map)

TABLE IV
NETWORK AND TRAINING PARAMETERS OF LSTM-CNN

Parameter	Value
Output features of convolutional layers	4
Kernel size of convolutional layers	(3, 3)
Stride of convolutional layers	1
Padding of convolutional layers	1
Output size of average pooling layers	(4, 4)
Size of fully connected layers	64×128
Input features of LSTM	128
Hidden features of LSTM	64
Size of the last fully connected layer	8×20
Batch size	20
Optimizer	Adam
Learning rate	0.001
Weight decay regularization	0.005

and assumes an idle car only takes an order originating from the current grid. However, in a real system, orders are dispatched based on the broadcasting distance between passengers and drivers. Therefore, even though some grids may not have arrivals, the idle cars at these locations can still take orders from neighboring grids as long as the distance between the vehicles and passengers is close. We overcome this limitation by uniformly splitting the arrival rate of a grid into its neighbors after predicting the demands online. Note that Algorithm 2 is a weighted splitting algorithm, where α_c, α_n can be chosen based on the platform order dispatching policy and the grid radius. Mode discussions can be found in Section VI-B.

C. Reposition Cost

The LP may have multiple solutions. To break the ties, we add reposition cost to the problem formulation to reduce the travel distance of idle cars, which corresponds to lower potential repositioning fuel costs or the bonus that the platform needs to pay when a reposition task is completed. We also propose a new LP whose solution has a lower travel cost while maintaining a similar order completion rate by adding a penalty term to the objective function. The new objective function is as follows:

$$(1a) - w_{tr} \sum_{i \neq j} \bar{e}_{ij} d_{ij} \quad (8)$$

where d_{ij} is the normalized distance ($d_{ij} \leq 1$) between region i and region j , and $w_{tr} \geq 0$ is a parameter that can be adjusted to balance these objectives. More details are presented in Section IV-G.

IV. PERFORMANCE ANALYSIS

In this section, we present extensive experiments that verify the performance and robustness of our proposed online repositioning policy.

A. Dataset

Our simulations were run on a platform that was constructed with a publicly available real-world dataset that was also utilized in the KDD Cup 2020 RL track competition¹. We used

the trip request data and the transition data in the dataset. The trip request data describes details of each order including each order id, the ride start and stop time, the pickup and drop-off locations, and the reward units. The transition data is based on the drivers' idle behavior in the real world, which is represented by the transition probability for each pair of origin and destination grids for each hour of the day. Before the simulation, we first need to summarize the statistical information from the dataset, that is, the arrival rates in each region and the transition rates of the incoming orders, in order to simulate the actual traffic flow. Detailed parameters can be found in Table V.

B. Simulation Platform

The platform² used in our simulation comprises $r = 20$ hexagon grids with a radius of around 600 m that are selected from the dataset's hot zones (high traffic regimes). The grid distributions are depicted in Fig. 4-(a). The orders are generated by sampling from the dataset. Passenger arrival rates are fictitious, based on Poisson distributions with time-varying parameters that update every 10 minutes. In order to impose spatial correlation, the arrival rates of some of the grids are determined as the average of those of their nearby grids. At every minute, the platform matches orders with available drivers and repositions the remaining idle drivers. The dispatching algorithm is fixed and unknown to the repositioning algorithm during the experiment. Orders can be transmitted to drivers within a certain broadcasting distance, which in our case is 1.2 kilometers. We chose $\alpha_c = 1, \alpha_n = 0.5$ in algorithm 2. Drivers remain where they are if they do not receive any orders and reposition recommendations. The time interval is from 1 : 00 p.m. to 8 : 00 p.m., and the number of drivers (vehicles) is fixed during this time period. Unless otherwise specified, the number of drivers is 300, and the repositioning rate used is 0.2. To make the simulation as close to a real system as possible, we assume that a driver can accept an order in the repositioning destination grid before getting there, and drivers who have completed repositioning must wait for 5 minutes before accepting a new reposition recommendation if no orders have been assigned to them. These parameters are chosen after consulting with researchers from a ride-hailing company.

In Algorithm 1, we set 10 minutes as one time slot. The lookahead time T is set to be 2 time slots (20 minutes) unless otherwise specified. We recalculated the repositioning routing matrix $q^*(t)$ at the beginning of every time slot. For the travel time $1/\mu_{ij}(t)$ and destination distribution $P_{ij}(t)$, we used the average of the historical data. For the arrival rate $\lambda_i(t)$, we used previous $M = 6$ time slots for the online prediction. The weight w_v was set to be 0.3 unless otherwise specified. The results of each experiment are averaged over 30 trials.

C. Baseline Algorithms

In the simulations, we compared our repositioning algorithms with the following policies.

²The source code of the simulator and our algorithm are available at: <https://github.com/honghaow/RL-for-Repositioning>

¹https://www.biendata.xyz/competition/kdd_didi/

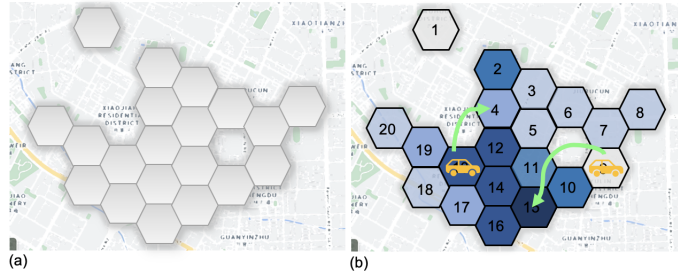


Fig. 4. (a) : An illustration of the grid map. (b) : An illustration of the repositioning process by our algorithm. The arrow denotes the repositioning direction. Darker color represents a higher demand.

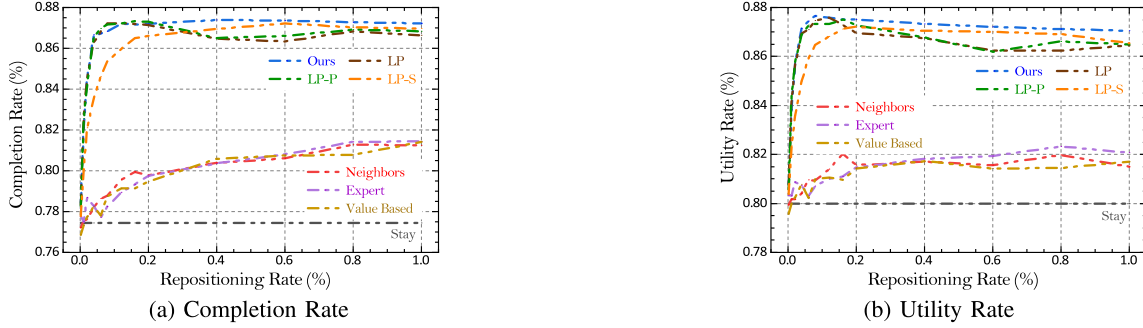


Fig. 5. Performance when varying the repositioning rate.

- Stay: Stay policy is simply no repositioning.
- Expert: A human expert policy extracted from the historical idle driver transition data which captures the collective intelligence.
- Neighbors: This is a heuristic policy that repositions idle drivers to neighboring grids uniformly at random.
- LP: This policy directly uses the solution of an LP without value function as the reposition policy. It uses the ground truth of expected arrival rates or predicted values of arrival rates (LP-P).
- LP-S: This policy uses the solution of the LP model of [1] under a stationary system assumption. It uses the ground truth of the expected arrival rates.
- Value Based: This stochastic policy will sample the reposition destination with probability proportional to the value function of neighbors' grids. (i.e., the method in [11].)

D. Repositioning Rate

In this subsection, we investigate the influence of repositioning rates on the total order completion rate and utility rate. The utility rate in online ride-sharing refers to the engagement of drivers and is computed as the ratio of working time to the total service time. This metric quantifies the level of satisfaction or benefit derived by both riders and drivers within the system. It is obvious that a higher utility rate indicates a more efficient repositioning algorithm. As shown in Fig. 5a, our algorithm outperforms the other baseline policies under almost all the different repositioning rates. For example, for repositioning rate $\eta = 0.2$, the completion rate of our algorithm increases by 9.3% compared with other non-LP-based policies, and 12.6% compared with no repositioning (Stay). Also, the order

completion rate increases when the repositioning rate increases until the “saturation” points. The saturation point of our algorithm is only 0.12 and the algorithm already achieves a completion rate as high as 0.873 by controlling only 10% vehicles which is possible and reasonable to be implemented in a real ride-hailing platform. However, for the LP without considering a non-stationary system (LP-S), the repositioning rate needs to be at least 60% in order to achieve a similar completion rate. LP-P has a similar performance as LP which means our prediction method can efficiently predict the true arrivals. In addition, including the value function in the objective further improves the performance. We also have a similar observation from the results in Fig. 5b when comparing the utility rate. Our algorithm can achieve a utility rate as high as 0.876 with only a limited repositioning rate of 0.12%.

E. Spatial Heterogeneity

As we discussed previously, the imbalance between regions may have a significant impact on the performance of the repositioning algorithms. In this subsection, we study the impact of spatial heterogeneity of the arrival rates on the performance of the repositioning algorithms. We manually move some arrivals from the 10 grids with lower arrival rates to the other 10 grids to increase spatial heterogeneity. We used the variance of the arrival rates of the 20 grids to quantify the spatial heterogeneity. Fig. 6(a) shows that as the spatial heterogeneity increases, the completion rate decreases for all algorithms, but at different rates. For example, for repositioning rate $\eta = 0.2$ and spatial variance 70, the Neighbors policy only has a completion rate of 0.587, but our algorithm can still achieve a completion rate of 0.786. Our algorithm performs better under two different repositioning

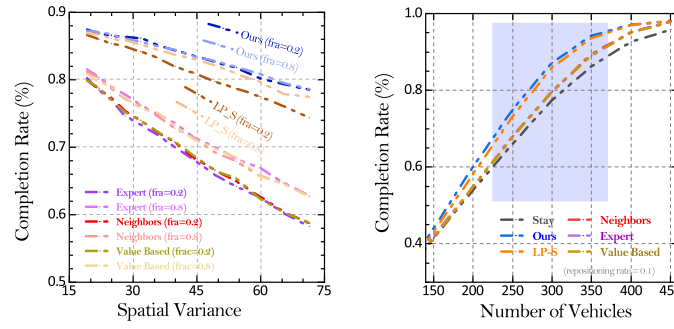


Fig. 6. (a) :Robustness of our algorithm with different repositioning rates; (b) : Completion Rate under different regimes.

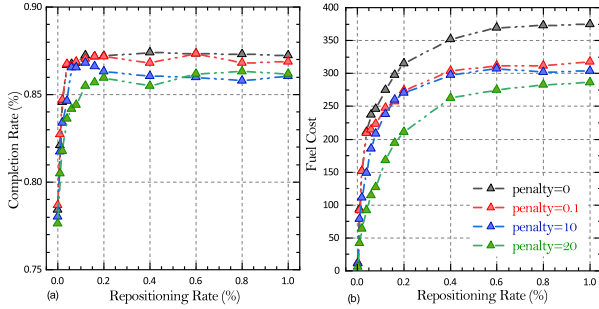


Fig. 7. Completion Rate under different travel cost penalties.

rates than LP-S which uses true arrival rates to obtain a solution.

F. Different Traffic Regimes

We study how the performance changes under different demand-supply gaps in this simulation. As shown in Fig. 6-(b), not surprisingly, the completion rate increases when the number of vehicles increases for all the algorithms. In fact, the number of vehicles determines the supply-demand gap of the system. When the number of vehicles is large, i.e., supply is enough, there is no need to do repositioning and all the reposition algorithms can have reasonably good performance. On the contrary, when the number of vehicles is small, i.e., supply is rare, there is also no need to do repositioning since roughly all the vehicles are busy all the time and there are no idle drivers in the system. This indicates that there exists a traffic regime (250 – 350 vehicles in our case) in which the reposition algorithms can play a significant role.

G. Repositioning Cost

As we mentioned in Section III-C, we also include the repositioning cost into the objective function when the LP has multiple feasible optimal solutions. Fig. 7 compares the travel (fuel) cost and the completion rate by varying the weight w_{tr} in (8). Simulation results show that by including a small penalty, the travel cost can be significantly reduced by up to 18.5% while maintaining a similar completion rate (penalty 0 v.s. penalty 0.1).

It is important to highlight the time efficiency of our algorithm as well, as it is capable of generating a policy within

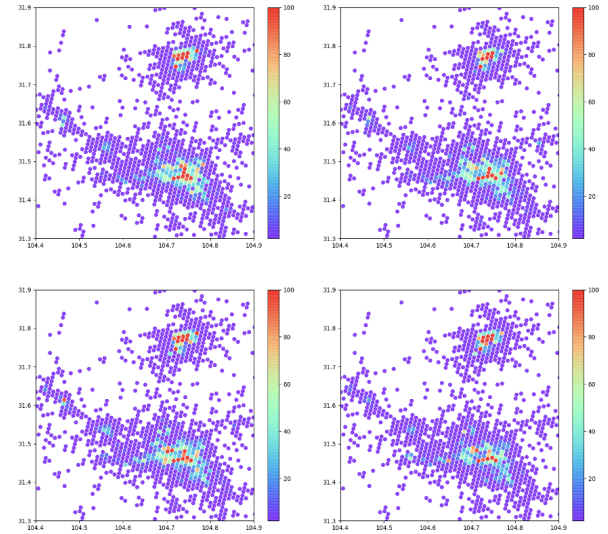


Fig. 8. Demand heat-map across different hours.

a matter of seconds. This time frame is significantly shorter than the one-time slot interval utilized in the simulation, which spans a duration of 10 minutes.

V. SIMULATION ON ONE OF THE WORLD'S LARGEST ONLINE RAID-HAILING PLATFORM

To further validate the practicality of our algorithm, we modify and evaluate our method on one of the largest ride-hailing platforms in the world using city-level data. The dataset is significantly larger and includes millions of ride-hailing orders. The demand distributions of the real-world data used in the simulator across different hours are shown in Fig. 8. Warmer colors represent higher demand quantities. It is easy to observe that imbalance commonly exists during the day. The details of both simulators can be found in Section VI-A. There are several key differences we would like to point out: The simulated area has 1,745 grids (city-level), which makes it more challenging to solve the LP problem and drivers are allowed to take orders even during repositioning. Since both features bring new difficulties for solving the LP, in order to have timely repositioning instructions for idle drivers we modified our repositioning policy as follows:

- **Community Detection:** When the scale of the city is too large to deal with, we apply the Louvain method ([20])

TABLE V
DETAILS OF SIMULATORS

Parameter	Simple Simulator	Complex Simulator
Number of Grids	20	1745
Number of Drivers	300	3000
Broadcasting Distance	1.2km	2km
Moving speed	3m/s	3m/s
Dispatch Frequency	1 mins	2 seconds
Reposition Frequency	1 mins	100 seconds
Waiting time between two repositionings	5 mins	5 mins
Allow being dispatched during reposition	False	True
Radius of each region	600m	600m

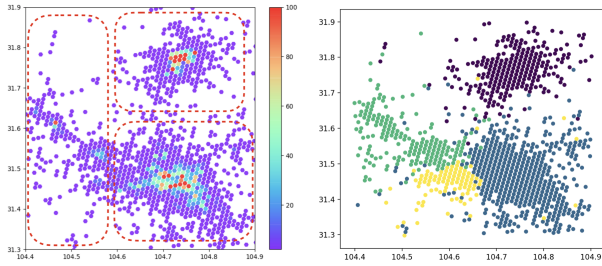


Fig. 9. Louvain method for community detection.

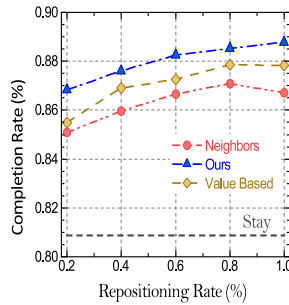


Fig. 10. Completion rate under different repositioning rates.

for community detection on the map. As shown in Fig. 9, the detection result on the right-hand side shows that the city is divided into four primary sections, which is consistent with the heat map on the left-hand side. We divided the city into three main parts (in the box) and each part is represented by a different LP. Then we'd be able to solve the parallel LPs and address the computation issue.

- **Reduce Number of Variables:** We restricted the repositioning distance to 2 hops which means idle drivers can only be repositioned to neighboring grids. This restriction significantly reduces the number of variables $\bar{e}_{ij}$ in the LP problem from 1745×1745 to 1745×19 .

As shown in Fig. 10, our method still outperforms other algorithms on the cite-level simulation environment. We adopted one of the state-of-art RL methods [7] to train the value function and compared it with their method (Value Based). Our algorithm increases the completion rate by 1%. Note that 1% is a significant improvement in ride-hailing systems both in terms of the number of additional passengers to be served and additional revenue to be earned. The improvement is comparable to those improvements reported in recent papers [7]

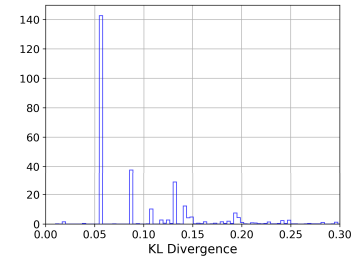


Fig. 11. Histogram of KL divergence between order distribution and the uniform distribution.

and [11], evaluated using data from real-world ride-hailing systems.

VI. ADDITIONAL SIMULATION DETAILS

A. Details of the Simulator

The details of the two simulators used in our paper are summarized below in Table V.

B. Uniformly Splitting Rule

In practice and our simulator, drivers are allowed to accept orders from neighbor grids. Our simulation uses $\alpha_c = \alpha_n = 1$ in the algorithm 2. The reason is that when a driver is located at a center of a grid, it can roughly accept orders from all the neighboring regions since the broadcasting distance is almost 3 times the radius of each region. In the following, we discuss how to choose the parameters.

We first collect the dispatch distribution of all the grids across one entire simulation and then calculate the KL divergence between the dispatch distribution of each grid and the uniform distribution. As shown in Fig. 11, we observe that the dispatch policy is close to a uniformly distributed policy, which means that a given order is dispatched to the drivers among the neighbors equally likely. That is the reason why we uniformly split the arrivals rate of each grid to all its neighbors but ensure that the overall arrival rate remains the same.

C. Lookahead Length

We would like to investigate the influence of different lookahead lengths. Fig. 12 shows that when the lookahead length increases, the completion rate tends to increase first, because of considering the system information of the near future, and then decrease, because of the increasing noise in the estimation when the planning horizon is too long. The optimal lookahead length is 30 minutes in our simulation.

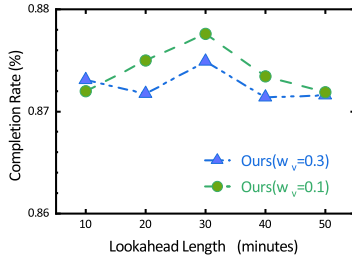


Fig. 12. Completion Rate when varying the controllable fractions and lookahead length.

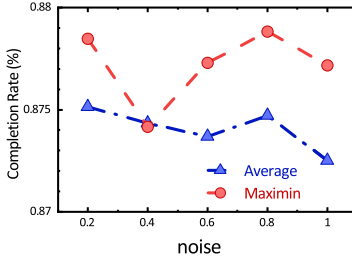


Fig. 13. Completion Rate when varying the noise level and lookahead length.

TABLE VI

PARAMETERS FOR TRAINING THE VALUE FUNCTION $V(j, s)$

Parameter	Value
Discount factor γ	0.95
Batch size	20000
Learning rate α	$1/N(j, s)$
Time horizon T_{end}	144

D. Maximin Objective Function

To elaborate on the advantage of the Maximin objective function. We compare our objective with the following objective function which maximizes the average completion rate over all the regions and T slots in [1] under the same constraints.

$$\max_{\bar{a}, \bar{f}, \bar{e}} \frac{1}{T} \sum_{u=0}^{T-1} \frac{\sum_{i=1}^r \lambda_i(t+u) \bar{a}_i}{\sum_{j=1}^r \lambda_j(t+u)} + w_v \sum_j V(j, t+T) \sum_i (\bar{e}_{ij} + \bar{f}_{ij}) \quad (9)$$

We add different levels of Gaussian noise to the arrival rates when sampling orders, in particular, the x-axis represents the variance of the level of noise. As shown in Fig. 13, we observe that although these two objectives both work well, the maximin objective is more robust to the noise.

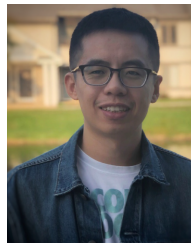
E. Parameters Used to Train the Value Function

The parameters that are used to train the value function $V(j, s)$ are shown in Table VI, where $N(j, s)$ is the number of times that the training process has visited the state (j, s) .

REFERENCES

- [1] A. Braverman, J. G. Dai, X. Liu, and L. Ying, "Empty-car routing in ridesharing systems," *Oper. Res.*, vol. 67, no. 5, pp. 1437–1452, Sep. 2019.
- [2] E. Özkan and A. R. Ward, "Dynamic matching for real-time ride sharing," *Stochastic Syst.*, vol. 10, no. 1, pp. 29–70, Mar. 2020.

- [3] R. Zhang and M. Pavone, "Control of robotic mobility-on-demand systems: A queueing-theoretical perspective," *Int. J. Robot. Res.*, vol. 35, nos. 1–3, pp. 186–203, Jan. 2016.
- [4] R. Iglesias, F. Rossi, K. Wang, D. Hallac, J. Leskovec, and M. Pavone, "Data-driven model predictive control of autonomous mobility-on-demand systems," in *Proc. IEEE Int. Conf. Robot. Autom. (ICRA)*, May 2018, pp. 6019–6025.
- [5] S.-F. Cheng, S. S. Jha, and R. Rajendram, "Taxis strike back: A field trial of the driver guidance system," in *Proc. 17th Int. Conf. Auto. Agents Multiagent Syst. (AAMAS)*, Stockholm, Sweden, vol. 15, Jul. 2018, pp. 577–584.
- [6] C. Riley, P. Van Hentenryck, and E. Yuan, "Real-time dispatching of large-scale ride-sharing systems: Integrating optimization, machine learning, and model predictive control," 2020, *arXiv:2003.10942*.
- [7] X. Tang et al., "A deep value-network based approach for multi-driver order dispatching," in *Proc. 25th ACM SIGKDD Int. Conf. Knowl. Discovery Data Mining*, 2019, pp. 1780–1790.
- [8] C. Mao, Y. Liu, and Z.-J.-M. Shen, "Dispatch of autonomous vehicles for taxi services: A deep reinforcement learning approach," *Transp. Res. C, Emerg. Technol.*, vol. 115, Jun. 2020, Art. no. 102626.
- [9] K. Lin, R. Zhao, Z. Xu, and J. Zhou, "Efficient large-scale fleet management via multi-agent deep reinforcement learning," in *Proc. 24th ACM SIGKDD Int. Conf. Knowl. Discovery Data Mining*, 2018, pp. 1774–1783.
- [10] J. Holler et al., "Deep reinforcement learning for multi-driver vehicle dispatching and repositioning problem," in *Proc. IEEE Int. Conf. Data Mining (ICDM)*, Nov. 2019, pp. 1090–1095.
- [11] Y. Jiao et al., "A deep value-based policy search approach for real-world vehicle repositioning on mobility-on-demand platforms," in *Proc. NeurIPS Deep RL Workshop*, 2020, pp. 1–10.
- [12] X. Tang et al., "Value function is all you need: A unified learning framework for ride hailing platforms," 2021, *arXiv:2105.08791*.
- [13] Z. Qin, H. Zhu, and J. Ye, "Reinforcement learning for ridesharing: A survey," in *Proc. IEEE Intell. Transp. Syst. Conf.*, Sep. 2021, pp. 2447–2454.
- [14] P. Afèche, O. Baron, J. Milner, and R. Roet-Green, "Pricing and prioritizing time-sensitive customers with heterogeneous demand rates," *Oper. Res.*, pp. 1184–1208, May 2019.
- [15] Z. Li, Y. Li, and L. Li, "A comparison of detrending models and multi-regime models for traffic flow prediction," *IEEE Intell. Transp. Syst. Mag.*, vol. 6, no. 4, pp. 34–44, Winter 2014.
- [16] H. Yao et al., "Deep multi-view spatial-temporal network for taxi demand prediction," in *Proc. AAAI Conf. Artif. Intell.*, Apr. 2018, vol. 32, no. 1, pp. 2588–2595.
- [17] G. Guo and T. Zhang, "A residual spatio-temporal architecture for travel demand forecasting," *Transp. Res. C, Emerg. Technol.*, vol. 115, Jun. 2020, Art. no. 102639.
- [18] T. Zhang and G. Guo, "Graph attention LSTM: A spatiotemporal approach for traffic flow forecasting," *IEEE Intell. Transp. Syst. Mag.*, vol. 14, no. 2, pp. 190–196, Mar. 2022.
- [19] G. Guo, W. Yuan, J. Liu, Y. Lv, and W. Liu, "Traffic forecasting via dilated temporal convolution with peak-sensitive loss," *IEEE Intell. Transp. Syst. Mag.*, vol. 15, no. 1, pp. 48–57, Jan. 2023.
- [20] V. D. Blondel, J.-L. Guillaume, R. Lambiotte, and E. Lefebvre, "Fast unfolding of communities in large networks," *J. Stat. Mech., Theory Exp.*, vol. 2008, no. 10, Oct. 2008, Art. no. P10008.



Honghao Wei (Member, IEEE) received the Ph.D. degree in electrical and computer engineering from the University of Michigan, Ann Arbor. He is currently a tenure-track Assistant Professor with the School of Electrical Engineering and Computer Science, Washington State University. His research interests include reinforcement learning, constrained reinforcement learning, and optimization and reinforcement learning, with applications in communication, social, and ride-sharing networks.



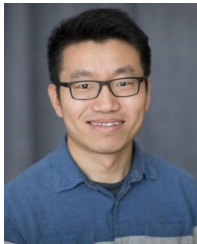
Zixian Yang received the B.E. and M.E. degrees from Sun Yat-sen University, China. He is currently pursuing the Ph.D. degree with the Department of Electrical Engineering and Computer Science, University of Michigan, Ann Arbor. His research interests include multi-armed bandit and reinforcement learning, with applications in multi-server scheduling, recommendation systems, communication, and ride-sharing networks.



Xiaocheng Tang received the Ph.D. degree from the Department of Industrial Engineering and Operations Research (IEOR), Lehigh University, PA, USA. He is currently a Research Scientist with Meta AI. His research interest lies broadly in machine learning, reinforcement learning, and large language modeling, with applications in recommendation systems and autonomous driving.



Xin Liu received the Ph.D. degree in electrical engineering from Arizona State University. He was a Post-Doctoral Research Fellow with the Department of Electrical Engineering and Computer Science, University of Michigan, Ann Arbor. He is currently a tenure-track Assistant Professor with the School of Information Science and Technology, ShanghaiTech University. His research interests lie broadly in stochastic modeling, analysis, and optimization, online learning and decision-making, and reinforcement learning, with applications in large-scale server systems, communication, and ride-sharing networks.



Zhiwei (Tony) Qin (Member, IEEE) received the B.Sc. degree from the University of British Columbia, Vancouver, and the M.S. and Ph.D. degrees from the Department of Industrial Engineering and Operations Research (IEOR), Columbia University, New York. He is currently a Principal Scientist with the Lyft Rideshare Laboratories. His research interest lies broadly in operations research, machine learning/deep learning, and big data analytics, with applications in smart transportation and e-commerce. He is an Associate Editor of the *ACM Journal on Autonomous Transportation Systems*.



Lei Ying (Fellow, IEEE) received the B.E. degree from Tsinghua University, Beijing, China, and the M.S. and Ph.D. degrees in electrical and computer engineering from the University of Illinois at Urbana-Champaign. He is currently a Professor with the Electrical Engineering and Computer Science Department, University of Michigan, Ann Arbor. His research is broadly in the interplay of complex stochastic systems and big data, including reinforcement learning, large-scale communication/computing systems for big-data processing, private data marketplaces, and large-scale graph mining. He is the Editor-at-Large of the *IEEE/ACM TRANSACTIONS ON NETWORKING* and an Associate Editor of the *Performance Evaluation* (Elsevier).